

COMP-421 Database System

School of Computer Science, McGill University

Winter 2025

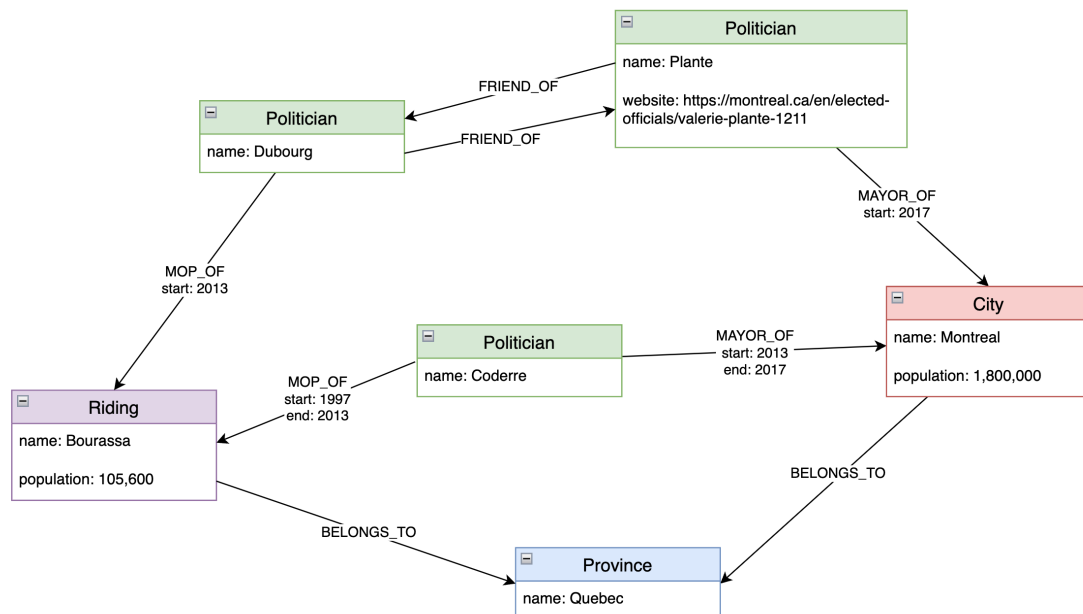
Jocelyne Li (261178604)

Assignment 3: Graph Databases and Query Evaluation

1 Part I: Graph Databases

1.1 Ex. 1 —Modeling Graph Databases vs. E/R (15 points)

Graph database:



Cypher query to find politicians who have been mayor of Montreal twice, with at least one other person serving as mayor in between:

```
MATCH (p:Politician)-[m1:MAYOR_OF]->(city:City {name: "Montreal"}),
      (p)-[m2:MAYOR_OF]->(city),
      (q:Politician)-[m_between:MAYOR_OF]->(city)
WHERE m1 <> m2
      AND m1.end < m_between.start AND m_between.end < m2.start
      AND p <> q
RETURN DISTINCT p.name AS PoliticianWhoWasMayorTwice;
```

Entity Sets and Primary Keys

- **Politician**(pid, name, website)
name is not unique, so pid is used as surrogate key
- **City**(cid, name, population)
name is not unique, so cid is primary key

- **Riding**(name, population)
name is unique, used as primary key
- **Province**(name)
name is unique

Relationship Sets and Keys

- **MayorOf**(pid, cid, start, end)
Between Politician and City, with start and end year
- **MoPOf**(pid, riding_name, start, end)
Between Politician and Riding, with start and end year
- **BelongsTo_City**(cid, province_name)
Each city belongs to one province
- **BelongsTo_Riding**(riding_name, province_name)
Each riding belongs to one province
- **Friendship**(pid1, pid2)
Symmetric relationship between two politicians

1.2 Ex. 2 —Cypher Queries (15 points)

1.(0 points) Find all 300-level courses.

```
MATCH (c:Course)-[:IS_OF_TYPE]->(t:Type {name: "300-level"})
RETURN DISTINCT c.name, c.title;
```

2.(0 points) Find all pairs of students that are either friends or roommates.

```
MATCH (s1:Student) -[r]->(s2:Student)
WHERE type(r) IN["FRIEND_OF", "ROOMMATES"]
RETURN DISTINCT s1.name AS Student1, s2.name AS Student2;
```

3.(0 points) Find all pairs of students that are roommates but not friends

```
MATCH (s1:Student)-[:ROOMMATES]->(s2:Student)
WHERE NOT (s1)-[:FRIEND_OF]->(s2) AND NOT (s2)-[:FRIEND_OF]->(s1)
RETURN DISTINCT s1.name AS Student1, s2.name AS Student2;
```

4.(5 points) Find all pairs of students that are friends and roommates.

```
MATCH (s1:Student)-[:FRIEND_OF]->(s2:Student),
      (s1)-[:ROOMMATES]->(s2)
RETURN DISTINCT s1.name AS Student1, s2.name AS Student2;
```

5.(5 Points) Find the names of the students who have taken courses at a different university than the one they study at.

```
MATCH (s:Student)-[:TAKES]->(c:Course)-[:GIVEN_AT]->(cu:University),
      (s)-[:STUDY_AT]->(su:University)
WHERE cu.name <> su.name
RETURN DISTINCT s.name;
```

6.(5 Points) Find pairs of students that take the same course and have a linkage through friend relationships (i.e., there is a path of edges labelled FRIEND OF that connects those two students).

```

MATCH (s1:Student)-[:TAKES]->(c:Course)<-[:TAKES]-(s2:Student)
WHERE s1 <> s2 AND EXISTS {
    MATCH path = (s1)-[:FRIEND_OF*]-(s2)
    RETURN path
}
RETURN DISTINCT s1.name AS Student1, s2.name AS Student2, c.name AS
    ↪ SharedCourse;

```

2 Part II: Query Evaluation

2.1 Ex. 3 —Index Sizes (10 points)

1.total number of data entries, the number of rids per data entry, and the size of a data entry

- **Total number of data entries:** Since amount has 3,000 distinct values, we have:

$$\text{Data entries} = 3,000$$

- **Number of RIDs per data entry:** There are 120,000 payments, distributed uniformly over 3,000 amount values:

$$\frac{120,000 \text{ payments}}{3,000 \text{ values}} = 40 \text{ RIDs per entry}$$

- **Size of a data entry:**

- Key (amount): INT = 4 bytes
- Each RID = 10 bytes
- Total size:

$$4 + (40 \times 10) = \boxed{404 \text{ bytes}}$$

2.the number of leaf pages and the height of the tree

- **Usable space per page:**

$$4,000 \times 0.75 = 3,000 \text{ bytes}$$

- **Data entries per page:**

$$\left\lfloor \frac{3,000}{404} \right\rfloor = 7$$

- **Leaf pages:**

$$\left\lceil \frac{3,000}{7} \right\rceil = \boxed{429}$$

- **Internal node entry size:**

- Key (INT) = 4 bytes, pointer = 6 bytes $\rightarrow$ entry = 10 bytes
- Entries per internal node:

$$\left\lfloor \frac{3,000}{10} \right\rfloor = 300$$

- **Internal pages:**

$$\left\lceil \frac{429 \text{ leaf pages}}{300} \right\rceil = 2$$

- **Root page:**

1 (always fits)

- **Total tree height:**

Root (1) $\rightarrow$ Internal (2) $\rightarrow$ Leaves (429) $\Rightarrow$ 3

2.2 Ex. 4 —Using indices (25 points)

```
SELECT *
FROM Payment
WHERE paydate = '2025-03-05' AND amount > 2750;
```

1. Indicate the access costs (in number of pages retrieved leading to I/O) for this query in each of the scenarios for the case that D=2025-03-05 and M=2750. Provide a simple calculation that shows how you have derived the values.

Assumptions and setup

- Total `Payment` records: 120,000
- Records are evenly distributed across 365 days:

$$\frac{120,000}{365} \approx 329 \text{ records on 2025-03-05}$$

- `amount` has 3,000 distinct values, uniformly distributed
- Fraction of payments with `amount > 2750`:

$$\frac{250}{3000} = \frac{1}{12}$$

- Estimated number of qualifying records:

$$329 \times \frac{1}{12} \approx 27.4 \approx 28 \text{ records}$$

- `Payment` table is stored in 1,000 pages

(a)(4 Points) you do not use any index.

Full scan of the **Payment** table.

$$\boxed{1000 \text{ I/Os}}$$

(b)(6 Points) you use the index on **paydate** (see above).

- Type II B⁺-tree, unclustered
- Leaf page for 2025-03-05 gives 329 RIDs
- We want $\frac{1}{12}$ of them $\Rightarrow$ 28 records
- In worst case, 28 different data pages (unclustered)

$$1 \text{ (leaf)} + 28 \text{ (pages)} = \boxed{29 \text{ I/Os}}$$

(c)(6 Points) you use the index on **amount** (see above).

- Values $> 2750 \Rightarrow$ 250 distinct **amount** values
- Each value: 40 RIDs $\Rightarrow 250 \times 40 = 10,000$ RIDs
- All must be checked for **paydate** = '2025-03-05'
- Each RID could point to a different page

$$250 \text{ (leaf pages)} + 10,000 \text{ (data pages)} = \boxed{10,250 \text{ I/Os}}$$

(d)(4 Points) you use both indexes.

- Intersect RIDs from both indexes in memory
- Same match count (≈ 28 records)
- Cost: 1 (**paydate** leaf) + 250 (**amount** leaves) + 28 data pages

$$1 + 250 + 28 = \boxed{279 \text{ I/Os}}$$

2.(0 Points) Calculate this also for general values of D and M.

Let:

- R = total records in **Payment** (120,000)
- V = distinct **amount** values (3,000)
- D = chosen date
- M = chosen threshold amount

Then:

$$\begin{aligned} \text{Records on } D &= \frac{R}{365}, \quad \text{Fraction satisfying } \text{amount} > M = \frac{V - M}{V} \\ \Rightarrow \text{Expected result size} &= \frac{R}{365} \times \frac{V - M}{V} \end{aligned}$$

3.(5 Points) What would be the access costs if the index on `paydate` was clustered? Give a short calculation that explains your answer.

In a clustered index, the records with the same `paydate` are stored physically close together in sorted order on disk. This means:

- There are approximately 329 payments made on 2025-03-05.
- Each payment record is estimated to be around 30 bytes in size.
- With a 75% page fill factor, each 4,000-byte page holds:

$$\left\lfloor \frac{3000}{30} \right\rfloor = 100 \text{ records per page}$$

- Therefore, 329 records span approximately:

$$\left\lceil \frac{329}{100} \right\rceil = 4 \text{ contiguous pages}$$

- Since we are only interested in those with `amount > 2750` (roughly 1/12 of them), only ≈ 28 records will match.
- These 28 qualifying records are likely concentrated within only **2 to 3 of those 4 pages** (not spread across all of them), due to physical locality from clustering.

Hence, the estimated cost is:

$$\text{Cost} \approx 1 \text{ (index leaf)} + 2\text{--}3 \text{ data pages} = \boxed{3\text{--}4 \text{ I/Os}}$$

2.3 Ex. 5 —Joins (15 Points)

Assume now a join between `Account` and `Payment` (WHERE `Account.accountid = Payment.srcaccountid`).

Assume an unclustered type II index on `accountid` of `Account`.

Indicate

Given:

- **Account:** 8,000 tuples $\Rightarrow$ 250 pages
- **Payment:** 120,000 tuples $\Rightarrow$ 1,000 pages
- Each `Payment.srcaccountid` matches 1 **Account**
- Unclustered index on `Account.accountid`: lookup cost = 2 I/Os (1 index page + 1 data page)

1.(0 points) the number of output tuples.

Each payment has one `srcaccountid`, and all are valid references:

$$\boxed{120,000 \text{ output tuples}}$$

2.(5 Points) the estimated number of I/O when using an index nested loop join.

- Outer = **Payment** (1000 pages), Inner = **Account** with index

$$\begin{aligned}\text{Cost} &= \text{Scan Payment} + \text{Index lookups per tuple} \\ &= 1000 + 120,000 \times 2 \\ &= \boxed{241,000 \text{ I/Os}}\end{aligned}$$

3.(5 Points) the estimated number of I/O when using block nested loop join and **Account** is the outer relation.

- Outer = **Account** (250 pages), Inner = **Payment** (1000 pages), $B = 52$

$$\left\lceil \frac{250}{B-2} \right\rceil = \left\lceil \frac{250}{50} \right\rceil = 5 \text{ iterations}$$

$$\text{Cost} = 250 + 5 \times 1000 = \boxed{5,250 \text{ I/Os}}$$

4.(5 Points) the estimated number of I/O when using sort merge join. Assume relations are not already sorted on the join attribute. Hint:- Remember we used a technique in class to reduce the cost of merge-join so that it is less than the sum of the individual costs of merge sort and join.

Assume neither **Account** nor **Payment** is sorted on the join attribute. We use the standard sort-merge join with pipelining, which allows us to combine the final sort pass with the merge join to reduce I/O cost.

Step-by-step cost:

- **Pass 0:** Read and write each relation once to produce sorted runs.

$$2 \times \text{AccountPages} + 2 \times \text{PaymentPages} = 2 \times 250 + 2 \times 1000 = 2500$$

- **Pass 1 and Merge Join (Pipelined):** Read the sorted runs again and perform the merge join simultaneously without writing the sorted output back to disk.

$$1 \times \text{AccountPages} + 1 \times \text{PaymentPages} = 250 + 1000 = 1250$$

- **Total I/O Cost:**

$$2500 + 1250 = \boxed{3750 \text{ I/Os}}$$

Note: The optimization here avoids an extra read/write step after sorting, by performing the merge join as we read the sorted data.